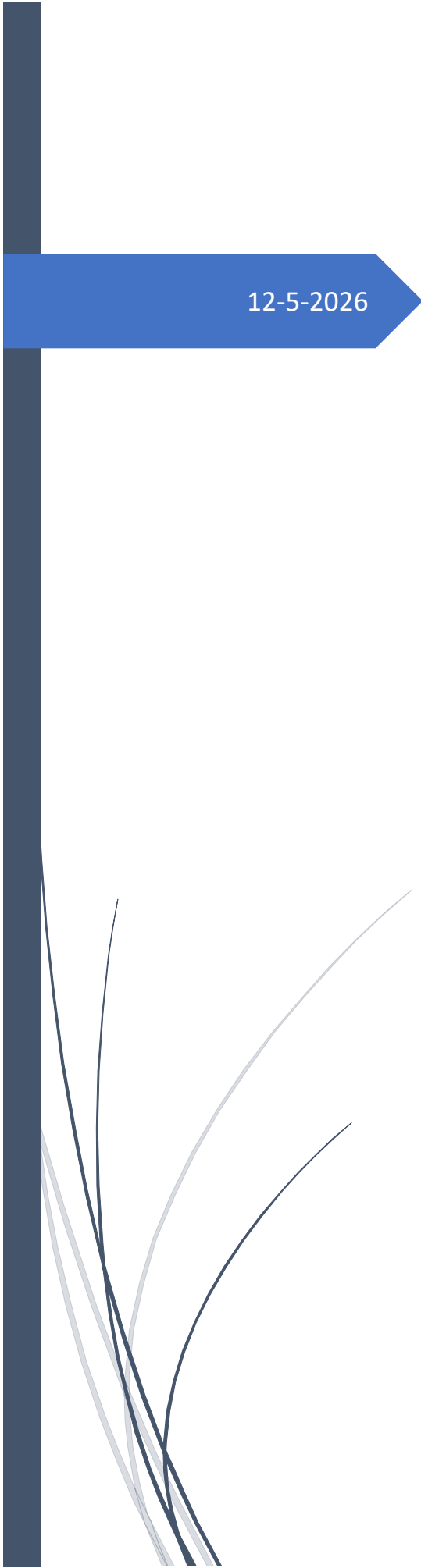




12-5-2026

# Explicación del desarrollo de la solución

Bio Vet - UTS

María Fernanda Gallo Serrano  
Gabriel Fernando Villar Suarez

Programa: Programación Java  
UNIDADES TECNOLÓGICAS DE SANTANDER  
2026

## Explicación del desarrollo de la solución:

### (BioVetUTS)

La solución se desarrolló utilizando el framework Spring Boot, integrando Spring Security para la gestión de autenticación y Thymeleaf como motor de plantillas para la interfaz de usuario. A continuación, detallaremos la arquitectura por capas aplicada en el proyecto:

#### 1. Capa de datos (Persistencia y Modelo)

Se implementó un modelo de datos basado en el diagrama de clases y el esquema de base de datos MYSQL existente.

- **Entidades (JPA):** Se crearon las clases Usuarios y Rol utilizando anotaciones de persistencia (@Entity). Se configuraron los atributos para que coincidan con la base de datos veterinaria\_uts, utilizando tipos de datos Integer para las llaves primarias (id\_usuario, id\_rol) para mantener la integridad con las llaves foráneas ya existentes.
- **Repositorios:** Se definió la interfaz UsuarioRepository extendiendo de JpaRepository. Esta permite realizar consultas automáticas a la base de datos, incluyendo el método personalizado findByUsername para buscar usuarios durante el proceso de login.

#### 2. Capa de seguridad (Autenticación)

Es el núcleo del requerimiento y se dividió en dos componentes principales:

- **Servicio de detalles de usuario:** Se creó la clase UserDetailsCustomService que implementa la interfaz UserDetailsService de Spring Security. Este componente es el encargado de consultar la base de datos, validar si el usuario existe y extraer su contraseña y su rol (autoridad) para permitirle el acceso.
- **Configuración de seguridad:** En la clase SecurityConfig, se definieron las políticas de acceso:
  - Se estableció que la ruta /login y los recursos estáticos (CSS) son públicos.
  - Se configuró que cualquier otra ruta (como /home) requiere una sesión activa.
  - Se definió un PasswordEncoder de tipo NoOpPasswordEncoder para validar las contraseñas almacenadas en texto plano en la base de datos del laboratorio.

#### 3. Capa de control y vista (Frontend)

- **Controladores:** Se implementó LoginController para gestionar el flujo de navegación entre la página de inicio de sesión y el panel principal (/home) tras el éxito de la autenticación.
- **Motor de plantillas (Thymeleaf):** Se diseñaron las vistas en HTML5 integrando Bootstrap para el diseño visual. Se utilizó Thymeleaf para:
  - Procesar el envío del formulario de login mediante th:action.
  - Gestionar el cierre de sesión (Logout) de forma segura.
  - Mostrar información dinámica una vez el usuario accede al sistema.

#### 4. Conexión y despliegue

- **Configuración:** En el archivo `application.properties`, se vinculó el proyecto con el servidor local de base de datos (XAMPP/MySQL), desactivando la modificación automática de tablas (`ddl-auto=none`) para proteger la estructura previa de la veterinaria.

| Rol        |           |
|------------|-----------|
| Usuario    | admin_uts |
| Contraseña | 12345     |